claude-windows / 9b717b4b-673e-43c5-b3f4-33099e782c45

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\9b717b4b-673e-43c5-b3f4-33099e782c45\subagents\workflows\wf_711ef326-a91\agent-ae497df5317bbd3bb.jsonl`
- SHA-256: `d2a6535b7cf4c74de0aabd510a16b597d729f707a964851bbf2bb8e8e044ac2b`
- Source modified: `2026-06-08T11:35:27+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf_711ef326-a91`
- session_id: `9b717b4b-673e-43c5-b3f4-33099e782c45`

Transcript

1. user (2026-06-08T11:34:33.025Z)

Read C:\Users\avidu\Projects\rally-annotator\vlc\rally_annotator.lua in full. Review ONLY for Lua scoping and VLC descriptor-scan load safety. CRITICAL CONSTRAINT: VLC scans descriptor() in a restricted sandbox that lacks base globals like ipairs/pairs; therefore the TOP-LEVEL chunk must call NO global function (only literals, local/global function DEFINITIONS, and operators). The two ipairs loops were already moved into activate(). Verify the v1.4 additions did not reintroduce any top-level global call. Verify ordering: planned_next_number() and refresh_next_field() are defined BEFORE their first use (refresh_buttons, save_rally, create_dialog, sync_to_current_video); w_next and w_help are declared (as locals) before any function that captures them; no use of a local before its definition that would capture a nil upvalue. Verify removing the help_visible variable left no dangling reference (grep for help_visible). Report concrete defects only.

2. assistant (2026-06-08T11:34:35.434Z)

I'll read the file in full and review it for the specified Lua scoping and VLC descriptor-scan load safety concerns.

3. user (2026-06-08T11:34:36.503Z)

```
1       --[[ rally_annotator.lua  --  VLC Lua EXTENSION (v1.4)
2
3         Rally Annotator for NET-SEPARATED RACQUET SPORTS
4         (badminton · tennis · table tennis · pickleball · padel)
5
6         While watching a match in VLC, mark each rally's START / END and the point-stop
7         reason, and append one CSV row per rally. Pause/scrub freely, then click ? the
8         callback snapshots the exact playback time. Output is a plain CSV that ingests
9         directly into common rally-segmentation tooling.
10
11        Output CSV columns (times in decimal SECONDS):
12            rally_number,start_time,end_time,ending_reason,sport
13
14        WHAT'S NEW IN v1.3
15          - In-dialog HELP button: usage + an ending-reason decision guide, rendered
16            inside the extension's own dialog (see "WHERE HELP LIVES" below).
17          - Two-step commit: Mark END now ARMS a rally; you pick the (required) reason
18            and click "Save Rally" to write it. The reason is RESET after every save,
19            so it can never silently reuse the previous rally's reason.
20          - Editable Start/End fields (fine-tune the marked seconds before saving).
21          - "Recent rallies" list with Edit selected / Delete selected, plus Undo last.
22
23        WHERE HELP LIVES (VLC internals, verified against the 3.0.x source):
24          The empty "About <name>.lua" window under Tools > Plugins and extensions >
25          Add-ons is VLC's AddonInfoDialog; its body text ("Lua script") is a hardcoded
```

```
  constant supplied by VLC's filesystem addon scanner (modules/misc/addons/
  fsstorage.c) and CANNOT be set by an extension. The descriptor() shortdesc/
  description below DO show, but in a different place ? the "More information"
  dialog reached from the *Active Extensions* tab. For real, author-controlled
  help we therefore render it INSIDE this extension's dialog (the HELP button).

WHY a button-dialog (not an auto-pause hook):
  VLC 3.x can expose pause via the {"playing-listener"} capability
  (status_changed/playing_changed), but that callback is flaky on macOS
  (VLC #22778) and the input/meta listeners are broken in VLC 4.0 (#27558).
  A persistent dialog whose button callbacks SNAPSHOT the current playback
  time is portable, precise, and lets the rater pause/scrub before committing.
  So we declare NO listener capabilities.

UNITS: in VLC 3.x, vlc.var.get(input,"time") is in MICROSECONDS -> /1e6.

Widget grid args are (col, row, colspan, rowspan), 1-indexed.

Install (Windows): copy this file to
    %APPDATA%\vlc\lua\extensions\rally_annotator.lua
  macOS:  ~/Library/Application Support/org.videolan.vlc/lua/extensions/
  Linux:  ~/.local/share/vlc/lua/extensions/
then VLC > Tools > Plugins and extensions > Reload extensions (or restart),
then enable it from the View menu.

MIT licensed. https://github.com/avidullu/rally-annotator
]]

---------------------------------------------------------------------------
-- Extension registration
---------------------------------------------------------------------------
function descriptor()
  return {
    title       = "Rally Annotator",
    version     = "1.4",
    author      = "Avi Dullu",
    url         = "https://github.com/avidullu/rally-annotator",
    shortdesc   = "Mark rally start/end + a point-ending reason to a CSV (net-separated
racquet sports)",
    description =
        "Mark each rally's START and END while you watch, tag WHY the point ended "
     .. "(winner / forced_error / unforced_error / service_fault / let / other), and
append "
     .. "one CSV row per rally next to the video "
     .. "(rally_number,start_time,end_time,ending_reason,sport; decimal seconds). "
     .. "Two-step Save (Mark END, then Save Rally) with a REQUIRED, non-sticky reason; "
     .. "editable times; edit or delete recent rallies. "
     .. "For badminton/tennis/table-tennis/pickleball/padel. "
     .. "Click the HELP button inside the dialog for usage + an ending-reason decision
guide.",
    capabilities = {}   -- button-click model: no listeners needed
  }
end

---------------------------------------------------------------------------
-- Config / constants
---------------------------------------------------------------------------
-- Net-separated racquet sports share a forced/unforced-error point-stop taxonomy.
local REASON_PLACEHOLDER = "-- choose reason --"
local REASONS = {
  "winner", "forced_error", "unforced_error", "service_fault", "let", "other"
}
local SPORTS = {
  "badminton", "tennis", "table_tennis", "pickleball", "padel"
}
```

```
88      local HEADER = "rally_number,start_time,end_time,ending_reason,sport\n"
89
90      -- reason/sport -> id lookups. POPULATED IN activate(), NOT here: VLC scans an
91      -- extension's descriptor() in a restricted Lua sandbox that lacks base globals
92      -- like ipairs, so calling ipairs at top level makes the whole extension fail to
93      -- load (it then never appears under the View menu).
94      local REASON_ID = {}
95      local SPORT_ID = {}
96
97      -- In-dialog help (rendered into the status panel by the HELP button). Basic HTML
98      -- only (<b>, <i>, <br>) for portability across the Qt and macOS dialog renderers.
99      local HELP_HTML = [==[
100     <b>Rally Annotator &mdash; how to use</b><br>
101     1. Pick the <b>Sport</b> (top). It stays set across rallies.<br>
102     2. When a rally begins, click <b>Mark START</b> (pause/scrub first for frame accuracy
&mdash; it snapshots the current playback time into the Start field).<br>
103     3. When the rally ends, click <b>Mark END</b>. You may fine-tune the Start/End seconds
by editing those fields directly.<br>
104     4. Choose the <b>Ending reason</b> (REQUIRED) and click <b>Save Rally</b> &mdash; one
CSV row is written next to the video.<br>
105     5. The reason RESETS after every save, so it is never reused by accident.<br>
106     6. <b>Recent rallies</b>: select a row, then <b>Edit selected</b> (loads it back into
the fields) or <b>Delete selected</b>. <b>Undo last</b> removes the most recent row (the button
shows which, e.g. <i>Undo last (#7)</i>), or clears an in-progress mark, or cancels an edit.<br>
107     7. <b>Resuming later:</b> labels are saved to the CSV next to the video as you go. Re-
open the SAME video and enable the extension &mdash; it reloads your existing rallies and
continues numbering. Set the <b>Next rally #</b> field (top right) to resume numbering from any
value (e.g. restart at 1, or continue from 50); it auto-advances after each save. If you switch
videos with this dialog open, click <b>Refresh</b> to load the current video's rallies. (The
video's playback position is not restored &mdash; scrub to where you stopped.)<br>
108     <br>
109     <b>Ending reasons &mdash; what they mean (pick the one that says WHY the rally
ended).</b><br>
110     All reasons except <i>winner</i> are charged to the side that LOST the rally.<br>
111     &bull; <b>winner</b> &mdash; the last shot landed IN and was not returned (opponent
couldn't reach it, or only waved at it). A clean untouched serve (ace) counts here.<br>
112     &bull; <b>forced_error</b> &mdash; the loser MISSED (out, or into the net) while UNDER
PRESSURE: stretched, rushed, jammed, or handling the opponent's pace/spin/depth. They were made
to miss.<br>
113     &bull; <b>unforced_error</b> &mdash; the loser MISSED a routine shot they had time AND
position to make, with little or no pressure. They gave it away.<br>
114     &bull; <b>service_fault</b> &mdash; the point ended on the SERVE: serve into the net,
out of the service box, illegal action/foot fault, or a double fault (tennis/padel).<br>
115     &bull; <b>let</b> &mdash; the rally is REPLAYED under the rules (no point): e.g. a
tennis/table-tennis serve that clips the net and is otherwise good, or outside interference.<br>
116     &bull; <b>other</b> &mdash; anything else (occluded footage, injury/retirement, penalty,
hindrance). Use sparingly.<br>
117     <br>
118     <b>Quick cases</b><br>
119     &bull; Ball/shuttle lands <b>OUT</b> (past the baseline / outside the lines): it is the
hitter's miss &mdash; <b>forced</b> if they were under pressure, <b>unforced</b> if it was a
comfortable ball. (OUT is NOT automatically forced, and never a winner for the opponent.)<br>
120     &bull; <b>Into the net</b> during a rally: same &mdash; forced if pressured, unforced if
routine. A <b>serve</b> into the net is <b>service_fault</b>.<br>
121     &bull; <b>Net-cord that dribbles over and lands in</b>: live and good &mdash; usually a
<b>winner</b> if unreachable, else judge what happens next. On the SERVE it varies by sport:
tennis/table-tennis = <b>let</b> (replay); badminton net-tick that passes over and lands in =
play on (but a serve shuttle CAUGHT on the net = service_fault); pickleball (2026) = play
on.<br>
122     &bull; <b>Unsure forced vs unforced?</b> Default to <b>unforced</b> &mdash; only mark
forced when you can point to the specific pressure.<br>
123     <br>
124     Output: <i>&lt;video&gt;.rallies.csv</i> next to the video. Full guide:
docs/ENDING_REASONS.md in the repo.<br>
125     Click <b>Hide help</b> to return to the status panel.
```

```
126     ]==]
127
128     ---------------------------------------------------------------------------
129     -- State
130     ---------------------------------------------------------------------------
131     local d                 -- the single dialog (one per extension)
132     local w_status          -- status label widget (HTML/rich-text); also hosts help
133     local w_reason          -- ending_reason dropdown widget
134     local w_sport           -- sport dropdown widget
135     local w_start           -- start-time text input (editable seconds)
136     local w_end             -- end-time text input (editable seconds)
137     local w_list            -- recent-rallies list widget
138     local w_save            -- "Save Rally" / "Save changes" button (relabeled by state)
139     local w_undo            -- "Undo last" button (relabeled to show which row it removes)
140     local w_next            -- "Next rally #" text input (lets you resume numbering
anywhere)
141     local w_help_btn        -- the Help button (relabeled "Hide help" when open)
142     local w_help            -- the help panel widget; nil when hidden (toggled via
del_widget)
143
144     local rows = {}         -- in-memory rally rows: { n, s, e, reason, sport, [extra] }
145     local mode = "new"      -- "new" (marking a fresh rally) or "edit" (editing a row)
146     local edit_index = nil  -- index into rows when mode == "edit"
147     local out_path          -- resolved CSV path (set on activate; re-resolved by Refresh)
148
149     ---------------------------------------------------------------------------
150     -- Helpers (declared before the global callbacks that capture them)
151     ---------------------------------------------------------------------------
152
153     -- Current playback time in SECONDS, or nil if nothing is playing.
154     local function now_seconds()
155       local input = vlc.object.input()
156       if not input then return nil end
157       local t_us = vlc.var.get(input, "time")   -- MICROSECONDS in VLC 3.x
158       if not t_us then return nil end
159       return t_us / 1000000.0
160     end
161
162     -- mm:ss.mmm for display only.
163     local function fmt_clock(s)
164       if not s then return "--:--" end
165       if s < 0 then s = 0 end
166       local m   = math.floor(s / 60)
167       local sec = s - m * 60
168       return string.format("%d:%06.3f", m, sec)
169     end
170
171     -- Minimal HTML-escape so paths/messages render literally in the rich-text label.
172     local function esc(text)
173       text = tostring(text or "")
174       text = text:gsub("&", "&amp;")
175       text = text:gsub("<", "&lt;")
176       text = text:gsub(">", "&gt;")
177       return text
178     end
179
180     -- Best-effort path to the currently playing media file (decoded from URI).
181     local function current_media_path()
182       local input = vlc.object.input()
183       if not input then return nil end
184       local item = vlc.input.item()           -- VLC 3.x: vlc.input.item()
185       if not item then return nil end
186       local uri = item:uri()                   -- item:uri()
187       if not uri or uri == "" then return nil end
188       local p = uri
```

```
    -- Strip the scheme. Windows file URIs look like file:///C:/dir/clip.mp4;
    -- POSIX ones like file:///home/user/clip.mp4 -- keep the POSIX leading "/".
    if p:match("^file:///%a:") then
      p = p:gsub("^file:///", "")            -- file:///C:/... -> C:/...
    else
      p = p:gsub("^file://", "")             -- file:///home/... -> /home/...
    end
    -- percent-decode (e.g. %20 -> space) BEFORE separator normalization
    p = p:gsub("%%(%x%x)", function(h) return string.char(tonumber(h, 16)) end)
    if p:match("^%a:") then                  -- Windows drive path -> backslashes
      p = p:gsub("/", "\\")
    end
    return p
  end

  -- Resolve a sensible default output path: next to the video if we can, else
  -- the user's home (USERPROFILE on Windows, HOME elsewhere).
  local function resolve_out_path()
    local media = current_media_path()
    if media then
      local dir, stem = media:match("^(.*[\\/])([^\\/]-)%.?[^\\/%.]*$")
      if dir and stem and stem ~= "" then
        return dir .. stem .. ".rallies.csv"
      end
      local d2 = media:match("^(.*[\\/])")
      if d2 then return d2 .. "rally_labels.csv" end
    end
    local home = os.getenv("USERPROFILE") or os.getenv("HOME") or "."
    local sep = home:find("\\") and "\\" or "/"
    if home:sub(-1) ~= sep then home = home .. sep end
    return home .. "rally_labels.csv"
  end

  -- Load all rally rows from the CSV into `rows` (header + blank lines skipped).
  -- Forgiving parser: any leading-integer line is a rally row; extra columns are
  -- preserved verbatim so we never drop richer metadata on rewrite.
  local function load_rows()
    rows = {}
    local f = io.open(out_path, "r")
    if not f then return end
    for line in f:lines() do
      line = line:gsub("\r$", "")
      if line:match("%S") then
        local parts = {}
        for field in (line .. ","):gmatch("([^,]*),") do parts[#parts + 1] = field end
        local n = tonumber(parts[1])
        if n and n == math.floor(n) then
          local row = {
            n      = math.floor(n),
            s      = tonumber(parts[2]) or 0,
            e      = tonumber(parts[3]) or 0,
            reason = (parts[4] ~= nil and parts[4] ~= "") and parts[4] or "other",
            sport  = parts[5] or "",
          }
          if #parts > 5 then
            local ex = {}
            for i = 6, #parts do ex[#ex + 1] = parts[i] end
            row.extra = table.concat(ex, ",")
          end
          rows[#rows + 1] = row
        end
      end
    end
    f:close()
  end
```

```
254
255     -- Rewrite the whole CSV from `rows` (header once). We write a temp file and swap
256     -- it into place so the live CSV is never lost if a step fails: rename is atomic on
257     -- POSIX (and succeeds for a brand-new file on Windows); when Windows can't rename
258     -- over an existing file we stash the original as .bak and roll back on error.
259     local function save_all()
260       local tmp = out_path .. ".tmp"
261       local f, err = io.open(tmp, "w")
262       if not f then return false, ("cannot open CSV for write: " .. tostring(err)) end
263       f:write(HEADER)
264       for _, r in ipairs(rows) do
265         local line = string.format("%d,%.3f,%.3f,%s,%s", r.n, r.s, r.e, r.reason, r.sport)
266         if r.extra and r.extra ~= "" then line = line .. "," .. r.extra end
267         f:write(line .. "\n")
268       end
269       f:close()
270       -- Atomic on POSIX; also succeeds on Windows when out_path doesn't exist yet.
271       if os.rename(tmp, out_path) then return true end
272       -- Windows overwrite path: keep a backup so the original is never lost.
273       local bak = out_path .. ".bak"
274       os.remove(bak)
275       local stashed = os.rename(out_path, bak)        -- move original aside (nil if none)
276       local ok, rerr = os.rename(tmp, out_path)
277       if not ok then
278         if stashed then os.rename(bak, out_path) end  -- roll back
279         return false, ("cannot replace CSV: " .. tostring(rerr))
280       end
281       os.remove(bak)
282       return true
283     end
284
285     -- Next monotonic rally_number (max existing + 1); keeps numbering across reopen.
286     local function next_rally_number()
287       local maxn = 0
288       for _, r in ipairs(rows) do if r.n > maxn then maxn = r.n end end
289       return maxn + 1
290     end
291
292     -- Read a text-input widget as a number, or nil if blank/non-numeric.
293     local function get_field_num(w)
294       if not w then return nil end
295       local t = w:get_text()
296       if not t then return nil end
297       t = t:gsub("%s+", "")
298       if t == "" then return nil end
299       return tonumber(t)
300     end
301
302     -- The number the NEXT new rally will get. Defaults to next_rally_number(), but the
303     -- user can type any value into the "Next rally #" field to resume/insert anywhere.
304     local function planned_next_number()
305       local v = get_field_num(w_next)
306       if v and v >= 1 then return math.floor(v) end
307       return next_rally_number()
308     end
309
310     -- Reset the "Next rally #" field to the natural next number for the current CSV.
311     local function refresh_next_field()
312       if w_next then w_next:set_text(tostring(next_rally_number())) end
313     end
314
315     local function index_of_rally(n)
316       for i, r in ipairs(rows) do if r.n == n then return i end end
317       return nil
318     end
```

```
319
320     -- First selected rally_number in the recent list, or nil. get_selection() returns
321     -- a { [id]=text } table in VLC 3.0.x (id is the rally_number we stored).
322     local function selected_rally_number()
323       if not w_list then return nil end
324       local sel = w_list:get_selection()
325       if type(sel) ~= "table" then return nil end
326       for id, _ in pairs(sel) do
327         return tonumber(id) or id
328       end
329       return nil
330     end
331
332     -- Dropdowns have no set_value in 3.0.x; the FIRST add_value after a clear() is
333     -- auto-selected. So we reset/select by clear()+rebuild with the desired value first.
334     local function rebuild_reason_placeholder()
335       if not w_reason then return end
336       w_reason:clear()
337       w_reason:add_value(REASON_PLACEHOLDER, 0)        -- id 0 = "not chosen"
338       for i, v in ipairs(REASONS) do w_reason:add_value(v, i) end
339     end
340
341     local function rebuild_reason_selected(sel)
342       if not w_reason then return end
343       w_reason:clear()
344       local id = REASON_ID[sel] or 99
345       w_reason:add_value(sel, id)                      -- first => shown selected
346       for i, v in ipairs(REASONS) do
347         if v ~= sel then w_reason:add_value(v, i) end
348       end
349     end
350
351     local function rebuild_sport_selected(sel)
352       if not w_sport then return end
353       if not sel or sel == "" then sel = SPORTS[1] end
354       w_sport:clear()
355       local id = SPORT_ID[sel] or 99
356       w_sport:add_value(sel, id)                       -- first => shown selected
357       for i, v in ipairs(SPORTS) do
358         if v ~= sel then w_sport:add_value(v, i) end
359       end
360     end
361
362     -- Repaint the recent-rallies list (last 12 rows).
363     local function refresh_list()
364       if not w_list then return end
365       w_list:clear()
366       local total = #rows
367       local starti = total - 11
368       if starti < 1 then starti = 1 end
369       for i = starti, total do
370         local r = rows[i]
371         w_list:add_value(
372           string.format("#%d   %s -> %s   [%s, %s]", r.n, fmt_clock(r.s), fmt_clock(r.e),
r.reason, r.sport),
373           r.n)
374       end
375       if d then d:update() end
376     end
377
378     -- Action-button labels reflect state. Save shows the rally number it will write
379     -- (or "Save changes (#N)" while editing). Undo last shows exactly which row it
380     -- will remove ("Undo last (#N)"), or that it will cancel an edit / clear a mark.
381     local function refresh_buttons()
382       if w_save then
```

```
        if mode == "edit" and edit_index and rows[edit_index] then
          w_save:set_text(string.format("Save changes (#%d)", rows[edit_index].n))
        else
          local s = get_field_num(w_start)
          local e = get_field_num(w_end)
          if s and e then
            w_save:set_text(string.format("Save Rally (#%d)", planned_next_number()))
          else
            w_save:set_text("Save Rally")
          end
        end
      end
      if w_undo then
        if mode == "edit" then
          w_undo:set_text("Undo last (cancel edit)")
        else
          local s = get_field_num(w_start)
          local e = get_field_num(w_end)
          if s or e then
            w_undo:set_text("Undo last (clear mark)")
          elseif #rows > 0 then
            w_undo:set_text(string.format("Undo last (#%d)", rows[#rows].n))
          else
            w_undo:set_text("Undo last")
          end
        end
      end
      if d then d:update() end
    end

    local function set_status(msg)
      if not w_status then return end
      local mode_line
      if mode == "edit" and edit_index and rows[edit_index] then
        mode_line = string.format(
          "Mode: EDITING rally #%d  (Save changes to commit, Undo last to cancel).",
rows[edit_index].n)
      else
        mode_line = "Mode: new rally  (Mark START, Mark END, choose reason, Save Rally)."
      end
      local last_line = ""
      if #rows > 0 then
        local lr = rows[#rows]
        last_line = string.format("Last row (what Undo removes): #%d  %s -&gt; %s
[%s]<br>",
          lr.n, esc(fmt_clock(lr.s)), esc(fmt_clock(lr.e)), esc(lr.reason))
      end
      w_status:set_text(string.format(
        "%s<br>%s<br>%sNow: %s  |  Rallies in CSV: %d<br>CSV: %s",
        esc(msg), esc(mode_line), last_line, esc(fmt_clock(now_seconds())), #rows,
esc(out_path)))
      if d then d:update() end
    end

    -- Clear the form back to a fresh "new rally" state (reason reset = non-sticky).
    local function reset_form()
      mode = "new"
      edit_index = nil
      if w_start then w_start:set_text("") end
      if w_end then w_end:set_text("") end
      rebuild_reason_placeholder()
      refresh_buttons()
      if d then d:update() end
    end

```

```
    -- Re-resolve the CSV path against whatever video is playing NOW and load its
    -- labels. Lets the user open a video later (or switch videos) and pick up exactly
    -- where they left off without toggling the extension off/on. Same video -> just
    -- re-read the CSV; a different video -> repoint and load that video's rallies.
    local function sync_to_current_video()
      if mode == "edit" then
        -- Reloading rows would invalidate edit_index; make the user resolve the edit first.
        refresh_list(); refresh_buttons()
        set_status("Finish (Save changes) or cancel (Undo last) the current edit before
refreshing.")
        return
      end
      local media = current_media_path()
      if not media then
        refresh_list(); refresh_buttons()
        set_status("No media is playing -- still using this CSV. Open the video, then click
Refresh.")
        return
      end
      local new_path = resolve_out_path()
      if new_path == out_path then
        load_rows()                  -- re-read in case the file changed on disk
        refresh_next_field()
        refresh_list(); refresh_buttons()
        set_status(string.format("Refreshed. %d rallies loaded for this video.", #rows))
        return
      end
      out_path = new_path            -- switched videos -> repoint + resume that file
      load_rows()
      reset_form()
      refresh_next_field()
      refresh_list()
      refresh_buttons()
      set_status(string.format(
        "Switched to the current video. Loaded %d existing rallies; next is #%d.", #rows,
next_rally_number()))
    end

    ---------------------------------------------------------------------------
    -- Button callbacks (VLC calls these on its main loop; kept global to be safe)
    ---------------------------------------------------------------------------
    function mark_start()
      local t = now_seconds()
      if not t then set_status("No media playing -- cannot mark START."); return end
      w_start:set_text(string.format("%.3f", t))
      refresh_buttons()
      set_status(string.format("START set @ %s. Play to the rally's end, then Mark END.",
fmt_clock(t)))
    end

    function mark_end()
      local t = now_seconds()
      if not t then set_status("No media playing -- cannot mark END."); return end
      w_end:set_text(string.format("%.3f", t))
      refresh_buttons()
      set_status(string.format("END set @ %s. Choose the Ending reason, then click Save
Rally.", fmt_clock(t)))
    end

    function save_rally()
      local s = get_field_num(w_start)
      local e = get_field_num(w_end)
      if not s then set_status("Set a START time first (click Mark START)."); return end
      if not e then set_status("Set an END time first (click Mark END)."); return end
      if e < s then s, e = e, s end   -- tolerate reversed marks
```

```
505       if e <= s then
506         set_status("END must be later than START (rally must be > 0s).")
507         return
508       end
509       local rid, reason = w_reason:get_value()   -- (id, text)
510       if not rid or rid == 0 or not reason or reason == "" or reason == REASON_PLACEHOLDER
then
511         set_status("Choose an Ending reason -- it is required (still on \"" ..
REASON_PLACEHOLDER .. "\").")
512         return
513       end
514       local _, sport = w_sport:get_value()
515       if not sport or sport == "" then sport = SPORTS[1] end
516
517       local msg
518       if mode == "edit" and edit_index and rows[edit_index] then
519         local r = rows[edit_index]
520         r.s, r.e, r.reason, r.sport = s, e, reason, sport
521         local ok, err = save_all()
522         if not ok then set_status("WRITE FAILED: " .. tostring(err)); return end
523         msg = string.format("Updated rally #%d: %s -> %s  [%s, %s].", r.n, fmt_clock(s),
fmt_clock(e), reason, sport)
524       else
525         local n = planned_next_number()
526         if index_of_rally(n) then
527           set_status(string.format("Rally #%d already exists -- set \"Next rally #\" to a
free number.", n))
528           return
529         end
530         rows[#rows + 1] = { n = n, s = s, e = e, reason = reason, sport = sport }
531         local ok, err = save_all()
532         if not ok then rows[#rows] = nil; set_status("WRITE FAILED: " .. tostring(err));
return end
533         if w_next then w_next:set_text(tostring(n + 1)) end   -- continue numbering from
here
534         msg = string.format("Saved rally #%d: %s -> %s  [%s, %s].", n, fmt_clock(s),
fmt_clock(e), reason, sport)
535       end
536
537       reset_form()       -- clears fields + resets the (required) reason to placeholder
538       refresh_list()
539       set_status(msg)
540     end
541
542     function edit_selected()
543       local n = selected_rally_number()
544       if not n then set_status("Pick a rally in the Recent list first, then Edit
selected."); return end
545       local idx = index_of_rally(n)
546       if not idx then set_status("Rally #" .. tostring(n) .. " not found (try Refresh).");
return end
547       local r = rows[idx]
548       mode = "edit"; edit_index = idx
549       w_start:set_text(string.format("%.3f", r.s))
550       w_end:set_text(string.format("%.3f", r.e))
551       rebuild_reason_selected(r.reason)
552       rebuild_sport_selected(r.sport)
553       refresh_buttons()
554       set_status(string.format(
555         "Editing rally #%d. Adjust Start/End/reason, then Save changes. (Undo last
cancels.)", r.n))
556     end
557
558     function delete_selected()
559       local n = selected_rally_number()
```

```
560       if not n then set_status("Pick a rally in the Recent list first, then Delete
selected."); return end
561       local idx = index_of_rally(n)
562       if not idx then set_status("Rally #" .. tostring(n) .. " not found (try Refresh).");
return end
563       table.remove(rows, idx)
564       local ok, err = save_all()
565       if not ok then set_status("WRITE FAILED: " .. tostring(err)); return end
566       reset_form()
567       refresh_list()
568       set_status(string.format("Deleted rally #%d. %d remaining.", n, #rows))
569     end
570
571     function undo_last()
572       if mode == "edit" then
573         reset_form(); refresh_list(); set_status("Edit cancelled.")
574         return
575       end
576       local s = get_field_num(w_start)
577       local e = get_field_num(w_end)
578       if s or e then
579         reset_form()
580         set_status("Cleared the in-progress START/END (nothing was written).")
581         return
582       end
583       if #rows == 0 then set_status("Nothing to undo."); return end
584       local last = rows[#rows]
585       rows[#rows] = nil
586       local ok, err = save_all()
587       if not ok then rows[#rows + 1] = last; set_status("WRITE FAILED: " .. tostring(err));
return end
588       refresh_list()
589       refresh_buttons()
590       set_status(string.format("Removed last rally #%d. %d remaining.", last.n, #rows))
591     end
592
593     function refresh_now()
594       sync_to_current_video()
595     end
596
597     function show_help()
598       -- Toggle a DEDICATED help panel by adding/removing a widget. In-place set_text on
599       -- the shared status widget did not repaint reliably in VLC (the panel got "stuck");
600       -- add_html + del_widget forces a real layout change that VLC renders every time.
601       if w_help then
602         if d then d:del_widget(w_help) end
603         w_help = nil
604         if w_help_btn then w_help_btn:set_text("Help") end
605       else
606         if d then w_help = d:add_html(HELP_HTML, 1, 14, 4, 8) end
607         if w_help_btn then w_help_btn:set_text("Hide help") end
608       end
609       if d then d:update() end
610     end
611
612     ---------------------------------------------------------------------------
613     -- Dialog construction
614     ---------------------------------------------------------------------------
615     local function create_dialog()
616       d = vlc.dialog("Rally Annotator")
617
618       d:add_label("Sport:", 1, 1, 1, 1)
619       w_sport = d:add_dropdown(2, 1, 2, 1)
620       for i, v in ipairs(SPORTS) do w_sport:add_value(v, i) end   -- badminton first =>
default
```

```
      w_help_btn = d:add_button("Help", show_help, 4, 1, 1, 1)

      d:add_label("Ending reason (required):", 1, 2, 1, 1)
      w_reason = d:add_dropdown(2, 2, 1, 1)
      d:add_label("Next rally #:", 3, 2, 1, 1)
      w_next = d:add_text_input("", 4, 2, 1, 1)

      d:add_label("Start (s):", 1, 3, 1, 1)
      w_start = d:add_text_input("", 2, 3, 1, 1)
      d:add_label("End (s):", 3, 3, 1, 1)
      w_end = d:add_text_input("", 4, 3, 1, 1)

      d:add_button("Mark START", mark_start, 1, 4, 1, 1)
      d:add_button("Mark END",   mark_end,   2, 4, 1, 1)
      w_save = d:add_button("Save Rally", save_rally, 3, 4, 2, 1)

      w_status = d:add_html("", 1, 5, 4, 2)   -- rich-text so multi-line (<br>) renders;
also hosts Help

      d:add_label("Recent rallies (select one, then Edit/Delete):", 1, 7, 4, 1)
      w_list = d:add_list(1, 8, 4, 4)

      d:add_button("Edit selected",   edit_selected,   1, 12, 1, 1)
      d:add_button("Delete selected", delete_selected, 2, 12, 1, 1)
      w_undo = d:add_button("Undo last", undo_last,     3, 12, 1, 1)
      d:add_button("Refresh",         refresh_now,     4, 12, 1, 1)

      rebuild_reason_placeholder()
      refresh_next_field()
      refresh_list()
      refresh_buttons()
      d:show()
      if #rows > 0 then
        set_status(string.format(
          "Resumed this video: %d existing rallies loaded (next is #%d). Click Help for the
guide.",
          #rows, next_rally_number()))
      else
        set_status("Ready. Pick the Sport, then mark rallies. Click Help for usage + the
ending-reason guide.")
      end
    end

    ---------------------------------------------------------------------------
    -- Lifecycle
    ---------------------------------------------------------------------------
    function activate()
      -- Build the id lookups here (full Lua environment), not at top level (see note
      -- where REASON_ID/SPORT_ID are declared -- the descriptor scan sandbox lacks ipairs).
      for i, v in ipairs(REASONS) do REASON_ID[v] = i end
      for i, v in ipairs(SPORTS) do SPORT_ID[v] = i end
      mode = "new"
      edit_index = nil
      w_help = nil
      out_path = resolve_out_path()
      load_rows()             -- continue an existing CSV (numbering + recent list)
      create_dialog()
    end

    function deactivate()
      if d then d:delete(); d = nil end
      w_help = nil
    end

    function close()
```

```
683       deactivate()
684       vlc.deactivate()
685     end
686
```

4. assistant (2026-06-08T11:34:38.920Z)

I've read the file in full. Let me verify the specific concerns by checking for the `help_visible` reference and confirming top-level safety.

5. user (2026-06-08T11:34:40.438Z)

No matches found

6. user (2026-06-08T11:34:41.659Z)

No matches found

7. assistant (2026-06-08T11:34:48.255Z)

Both grep checks come back clean. Let me carefully analyze the top-level chunk structure and ordering manually, since the file's structure matters precisely.

Let me verify the top-level statements that execute during the descriptor scan. The top-level (column-0, non-comment) statements are:
- Line 57: `function descriptor()` ? a function definition (no call)
- Lines 81-99: `local` assignments to literals/table constructors (`REASON_PLACEHOLDER`, `REASONS`, `SPORTS`, `HEADER`, `REASON_ID`, `SPORT_ID`)
- Line 99: `local HELP_HTML = [==[...]==]` ? long string literal
- Lines 131-147: `local d`, `local w_status`, etc. ? bare local declarations (no initializer, no call)
- Various `local function` and `function` definitions

Let me confirm there are no top-level table constructors that invoke a function, and double-check the `..` concatenations in the `local` constants and the `descriptor()` return (those are operators, safe). Let me look closely at the constant definitions and verify nothing at top level calls a function.

8. user (2026-06-08T11:34:49.415Z)

```
81:local REASON_PLACEHOLDER = "-- choose reason --"
82:local REASONS = {
85:local SPORTS = {
88:local HEADER = "rally_number,start_time,end_time,ending_reason,sport\n"
94:local REASON_ID = {}
95:local SPORT_ID = {}
99:local HELP_HTML = [==[
131:local d                 -- the single dialog (one per extension)
132:local w_status          -- status label widget (HTML/rich-text); also hosts help
133:local w_reason          -- ending_reason dropdown widget
134:local w_sport           -- sport dropdown widget
135:local w_start           -- start-time text input (editable seconds)
136:local w_end             -- end-time text input (editable seconds)
137:local w_list            -- recent-rallies list widget
138:local w_save            -- "Save Rally" / "Save changes" button (relabeled by state)
139:local w_undo            -- "Undo last" button (relabeled to show which row it removes)
140:local w_next            -- "Next rally #" text input (lets you resume numbering anywhere)
141:local w_help_btn        -- the Help button (relabeled "Hide help" when open)
142:local w_help            -- the help panel widget; nil when hidden (toggled via del_widget)
144:local rows = {}         -- in-memory rally rows: { n, s, e, reason, sport, [extra] }
145:local mode = "new"      -- "new" (marking a fresh rally) or "edit" (editing a row)
146:local edit_index = nil  -- index into rows when mode == "edit"
147:local out_path          -- resolved CSV path (set on activate; re-resolved by Refresh)
154:local function now_seconds()
```

```
163:local function fmt_clock(s)
172:local function esc(text)
181:local function current_media_path()
206:local function resolve_out_path()
225:local function load_rows()
259:local function save_all()
286:local function next_rally_number()
293:local function get_field_num(w)
304:local function planned_next_number()
311:local function refresh_next_field()
315:local function index_of_rally(n)
322:local function selected_rally_number()
334:local function rebuild_reason_placeholder()
341:local function rebuild_reason_selected(sel)
351:local function rebuild_sport_selected(sel)
363:local function refresh_list()
381:local function refresh_buttons()
413:local function set_status(msg)
435:local function reset_form()
449:local function sync_to_current_video()
615:local function create_dialog()
```

9. assistant (2026-06-08T11:35:01.609Z)

All top-level `local` statements are either bare declarations or assignments to literals/table constructors/long strings. None call a function. Now let me verify the ordering constraints precisely.

Ordering verification:

1. **`planned_next_number()` (defined line 304) and `refresh_next_field()` (defined line 311) before first use:**
   - `refresh_buttons` (defined line 381) ? uses `planned_next_number()` at line 389. 381 > 304 ?
   - `save_rally` (global function line 499) ? uses `planned_next_number()` at line 525. ?
   - `create_dialog` (line 615) ? uses `refresh_next_field()` at line 648. 615 > 311 ?
   - `sync_to_current_video` (line 449) ? uses `refresh_next_field()` at lines 465, 473. Both 449-478 > 311 ?

   Note: `planned_next_number` (304) calls `get_field_num` (293, defined earlier ?) and `next_rally_number` (286, earlier ?). `refresh_next_field` (311) calls `next_rally_number` (286, earlier ?). Good.

2. **`w_next` (line 140) and `w_help` (line 142) declared before any function capturing them:** All `local function` definitions start at line 154. All `function` (global) definitions are after. 140 and 142 < 154 ?. `w_next` is captured by `planned_next_number` (304), `refresh_next_field` (311), `refresh_buttons` (389/533 area), `save_rally`, `sync_to_current_video`, `create_dialog` ? all after 140 ?. `w_help` captured by `show_help` (597), `deactivate` (677), `activate` (671), `create_dialog` ? all after 142 ?.

3. **`w_help_btn` (line 141)** captured by `show_help` (597) and `create_dialog` (621) ? both after ?.

Now a subtle point worth checking. `planned_next_number` and `refresh_next_field` are **defined before `index_of_rally`** (line 315). But do any of them depend on functions defined later? Let me double check that no early `local function` captures a *later* local as an upvalue and is *called during the top-level chunk or descriptor scan*. None of these locals are called at top level ? they're only invoked from `activate()`/callbacks/dialog. So the upvalue-nil-capture risk only matters for call-time, which is fine in Lua (upvalues resolve to the variable, and the variable is assigned by the time `activate` runs).

Let me confirm there's no accidental top-level invocation (e.g., a stray `create_dialog()` or `activate()` call at column 0).

10. user (2026-06-08T11:35:02.872Z)

No matches found

11. assistant (2026-06-08T11:35:06.551Z)

No top-level function invocations exist. Let me also confirm the only top-level `..` concatenations are inside `descriptor()`'s return (which only runs when descriptor is called ? and string concatenation of literals is just the `..` operator, no global call), and verify the global `function` definitions don't have any issue. Let me check the global function definitions list.

12. user (2026-06-08T11:35:07.404Z)

```
57:function descriptor()
483:function mark_start()
491:function mark_end()
499:function save_rally()
542:function edit_selected()
558:function delete_selected()
571:function undo_last()
593:function refresh_now()
597:function show_help()
664:function activate()
677:function deactivate()
682:function close()
```

13. assistant (2026-06-08T11:35:10.479Z)

All global functions are definitions only (no top-level invocation). The descriptor concatenations at lines 64-72 are pure `..` operators on string literals ? operators, not global calls ? and only execute when `descriptor()` is invoked.

Let me do one final check on the two `ipairs` loops mentioned as moved into `activate()`, plus confirm all `ipairs`/`pairs` usages live inside function bodies (not top-level).

14. user (2026-06-08T11:35:11.809Z)

```
92:-- like ipairs, so calling ipairs at top level makes the whole extension fail to
264:  for _, r in ipairs(rows) do
288:  for _, r in ipairs(rows) do if r.n > maxn then maxn = r.n end end
316:  for i, r in ipairs(rows) do if r.n == n then return i end end
326:  for id, _ in pairs(sel) do
338:  for i, v in ipairs(REASONS) do w_reason:add_value(v, i) end
346:  for i, v in ipairs(REASONS) do
357:  for i, v in ipairs(SPORTS) do
620:  for i, v in ipairs(SPORTS) do w_sport:add_value(v, i) end   -- badminton first => default
666:  -- where REASON_ID/SPORT_ID are declared -- the descriptor scan sandbox lacks ipairs).
667:  for i, v in ipairs(REASONS) do REASON_ID[v] = i end
668:  for i, v in ipairs(SPORTS) do SPORT_ID[v] = i end
```

15. assistant (2026-06-08T11:35:15.985Z)

Every `ipairs`/`pairs` usage is inside a function body (indented, within `local function`/`function` blocks) ? lines 264, 288, 316, 326, 338, 346, 357 are inside helper functions; 620 inside `create_dialog`; 667-668 inside `activate`. None at top level. Lines 92 and 666 are comments.

The two id-lookup loops are correctly inside `activate()` (667-668). The SPORTS dropdown-population loop (620) is inside `create_dialog()`, called from `activate()`.

My review is complete. All constraints hold; no defects found.

16. user (2026-06-08T11:35:16.618Z)

Structured output provided successfully

17. assistant (2026-06-08T11:35:27.560Z)

Review complete. No defects found across all checked dimensions.

File reviewed: `C:\Users\avidu\Projects\rally-annotator\vlc\rally_annotator.lua` (686 lines, v1.4)

Top-level descriptor-scan safety ? clean. Every column-0 statement is either a function definition (`function`/`local function`), a bare `local` declaration, or a `local` assignment to a literal / table constructor / long-string (`[==[...]==]`). No top-level statement calls a global function:
- The 6 `local` constant assignments (lines 81-99) are all literals or empty/literal table constructors ? no `ipairs`/`pairs`/`string.*` etc.
- All 10 `ipairs`/1 `pairs` usages live inside function bodies (lines 264, 288, 316, 326, 338, 346, 357, 620, 667, 668); none at top level. Lines 92 and 666 are comments only.
- The two id-lookup loops are correctly inside `activate()` (lines 667-668); the SPORTS dropdown loop is inside `create_dialog()` (line 620).
- No stray top-level invocation exists (grep for `^name(` returned nothing).
- The `..` concatenations in `descriptor()` (lines 64-72) are operators, not global calls, and only run when `descriptor()` is invoked.

Ordering ? correct. `planned_next_number()` (def line 304) and `refresh_next_field()` (def line 311) are both defined before every use: `refresh_buttons` (389), `save_rally` (525), `create_dialog` (648), `sync_to_current_video` (465/473). Their own dependencies (`get_field_num` 293, `next_rally_number` 286) precede them.

Upvalue declarations ? correct. `w_next` (line 140) and `w_help` (line 142), plus `w_help_btn` (141), are declared as locals before the first `local function` (line 154) and before every function that captures them. No function is invoked during the top-level chunk, so there is no nil-upvalue capture at scan time.

`help_visible` removal ? clean. Grep for `help_visible` returns zero matches; no dangling reference remains. The help toggle now keys off `w_help` being nil vs. a widget handle (lines 601-608, 671, 679), which is consistent.